Project Based Learning (PBL) Report

Data Structures (BCS301)

B.Tech (CSE), III Sem, Section F

PARKING MANAGEMENT SYSTEM USING A SINGLY LINKED LIST IN C

Submitted by

Janvi Sengar (Roll No. 2502220100372)
Jaya Tripathi (Roll No. 2502220100377)
Khusboo Yadav (Roll No. 2502220100398)

Under the supervision of

Ms. Himani Sharma
Assistant Professor, Department of CSE



Submitted to
Department of Computer Science and Engineering & Allied Branches
ITS Engineering College, Greater Noida

2026-27

I.T.S Engineering College
A unit of Durga Charitable Society
46, Knowledge Park-III, Greater Noida, Gautam Budh Nagar (U.P)-201310
Ph: +91 (0120) 2331000, Toll Free : 180018008040
E-mail: engg.gn@its.edu.in Website: www.itsengg.edu.in

Certificate

This is to certify that the **Project Based Learning (PBL) report** entitled **"Parking Management System Using a Singly Linked List in C"** has been successfully carried out by the following students of **B.Tech (Computer Science & Engineering)** as a part of the **Project Based Learning** during the academic session **2026–2027**.

Submitted by:

Name: Janvi Sengar Roll No.: 2502220100372

Name: Jaya Tripathi Roll No.: 2502220100377

Name: Khusboo Yadav Roll No.: 2502220100398

The work has been completed under the guidance of **Ms. Himani Sharma**. The project is a part of continuous internal assessment under the PBL framework and reflects the students' effort in applying theoretical knowledge to solve real-world problems.

To the best of our knowledge, this work is **original** and has not been submitted elsewhere for any other academic purpose.

Ms. Himani Sharma

Mapping of the Project to Sustainable Development Goals (SDGs)

Project Name	Problem Description	SDG No. : SDG Name
Parking Management System Using a Singly Linked List in C	Manual parking registers make it slow and error-prone to know which slots are free, to find a parked vehicle and to work out the correct charge. Drivers lose time looking for space. A computerised, validated record of parked vehicles with automatic billing and persistent storage is needed.	SDG 11 : Sustainable Cities and Communities SDG 9 : Industry, Innovation and Infrastructure

Justification:

- **SDG 11 (Sustainable Cities and Communities):** a live record of occupied and free slots lets a parking lot make full use of its limited space and tells the attendant at once when the lot is full. This helps reduce vehicles circling in search of a slot and the congestion and fuel waste that follow.
- **SDG 9 (Industry, Innovation and Infrastructure):** it replaces a paper register with a small, dependable software tool, a step towards digital infrastructure for institutions that cannot afford large commercial systems.

CONTENTS

CERTIFICATE.....
MAPPING WITH SDG WITH JUSTIFICATION.....i
LIST OF FIGURES.....iii
LIST OF TABLES.....iii
ABSTRACT.....iv
CHAPTER 1: PROBLEM STATEMENT AND OBJECTIVES.....1
CHAPTER 2: METHODOLOGY / DESIGN.....2
CHAPTER 3: IMPLEMENTATION.....6
CHAPTER 4: RESULTS AND DISCUSSION.....11
CHAPTER 5: CONCLUSION.....13
REFERENCES.....14
APPENDIX: SOURCE CODE.....15

LIST OF FIGURES

Fig. 2.1: Singly linked list of vehicle nodes.....3
Fig. 2.2: Modular architecture of the system.....3
Fig. 3.1: Start-up screen showing the main menu.....6
Fig. 3.2: Parking vehicles.....7
Fig. 3.3: Displaying all parked vehicles.....7
Fig. 3.4: Searching for an existing and a missing vehicle.....7
Fig. 3.5: Removing vehicles and generating bills.....9
Fig. 3.6: Slot reuse, full parking lot and summary.....9
Fig. 3.7: Messages produced by invalid input.....10

LIST OF TABLES

Table 2.1: Functional requirements.....2
Table 2.2: Parking charges.....4
Table 2.3: Complexity of the operations (n = number of parked vehicles).....5
Table 4.1: Test cases and results.....11

ABSTRACT

Parking areas need to know which vehicles are inside, which slots are free and how much each driver must pay. This project presents a console-based Parking Management System in C. The vehicles that are currently parked are stored as a singly linked list whose nodes hold the vehicle number, owner name, vehicle type, slot number, entry time and a next pointer; nodes are created with `malloc()` and released with `free()`, so only vehicles that are actually parked use memory. A menu offers park, display, search, remove with bill, summary and exit. Input is read and parsed strictly, and duplicate vehicle numbers, invalid numbers, empty owner names, invalid vehicle types, parking in a full lot and removing an unknown vehicle are rejected. The bill is calculated from the entry time, rounded up to whole hours with a minimum of one hour. The list is saved to `parking.dat` after every change and validated when reloaded. All twenty test cases passed. Every operation takes time proportional to the number of parked vehicles (at most 20 slots) and the program uses O(n) space.

Keywords: Data Structures, Singly Linked List, Dynamic Memory, File Handling, C Programming.

CHAPTER 1
PROBLEM STATEMENT AND OBJECTIVES

1.1 INTRODUCTION

Parking lots at colleges, markets and offices must know which vehicles are inside, which slots are free and how much to charge. Paper registers make this slow and error-prone, and bills are often worked out by guesswork. This project is a console-based Parking Management System in C that stores each parked vehicle as a node of a singly linked list in dynamic memory and saves the list to a text file so that data survives between runs. The parking attendant uses a numbered menu to park, display, search and remove vehicles and to see a summary of the lot.

1.2 PROBLEM STATEMENT

To build a menu-driven parking management program in C that stores only the vehicles that are currently parked, allots a free slot to every vehicle, calculates the parking charge from the entry time, rejects invalid input without crashing, and keeps its data between sessions.

1.3 OBJECTIVES

1. Represent the parked vehicles as a dynamically allocated singly linked list of `struct Vehicle` nodes.
2. Implement park, display, search by vehicle number and remove operations.
3. Allot the lowest free slot to each vehicle, refuse entry when all slots are taken, and free the slot when the vehicle leaves.
4. Calculate the bill from the entry time and the vehicle type (two or four wheeler).
5. Validate all input and handle errors with clear messages.
6. Save data to a file after every change, reload it at start-up, and free all memory on exit.

1.4 SCOPE AND TOOLS

The system serves one attendant on one computer and one parking lot with 20 slots (`MAX_SLOTS`). Each parked vehicle has a unique vehicle number of letters and digits, an owner name, a type (two or four wheeler), a slot number and an entry time. Payment gateways, multi-level parking, advance booking, monthly passes, number plate recognition and a history of past bills are outside the scope. The program uses only `stdio.h`, `stdlib.h`, `string.h`, `ctype.h` and `time.h`; it was compiled with GCC using `-Wall -Wextra` without warnings.

CHAPTER 2
METHODOLOGY / DESIGN

2.1 REQUIREMENTS

The project followed the software development life cycle: problem formulation, requirement analysis, design, implementation and testing, built incrementally feature by feature.

Table 2.1: Functional requirements

ID	Requirement	Description
FR1	Park vehicle	Accept a unique vehicle number of letters and digits, a non-empty owner name and type 2 or 4; allot the lowest free slot, record the entry time and append to the list.
FR2	Display	Show all parked vehicles with slot, vehicle number, owner, type and entry time.
FR3	Search	Find a vehicle by its number and show its details.
FR4	Remove and bill	Print the bill (hours × rate, rounded up, minimum one hour), remove the vehicle and free its slot.
FR5	Summary	Show total, occupied and available slots and the count of two and four wheelers.
FR6	Persistence	Save to `parking.dat` after each change; load at start-up.
FR7	Validation	Reject invalid numbers, empty text, invalid types, duplicates, a full lot and invalid choices.

2.2 CHOICE OF DATA STRUCTURE

Vehicles arrive and leave in any order, so the number of records changes all the time and the vehicle that leaves is often neither the first nor the last one. A singly linked list was therefore chosen over an array. A node is created only when a vehicle enters and freed when it leaves, so memory use matches the number of vehicles actually inside, and removing a vehicle from any position needs only one pointer change instead of shifting elements. Its drawback, O(n) access by position, is not needed because vehicles are found by their number, and n can never exceed the 20 slots of the lot.

2.3 DATA DESIGN

Each parked vehicle is a `struct Vehicle`; the global pointer `head` holds the first node and is NULL for an empty lot. The slot number is stored in the node, but the number of free slots is not stored; it is computed as `MAX_SLOTS` minus the number of nodes, so two stored values can never disagree.

```
struct Vehicle
{
    char number[NUMBER_LEN];
    char owner[OWNER_LEN];
    int type;
    int slot;
    time_t entryTime;
    struct Vehicle *next;
};

struct Vehicle *head = NULL;
```

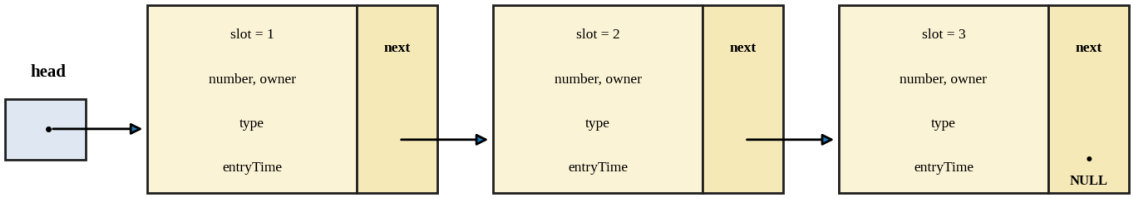


Fig. 2.1: Singly linked list of vehicle nodes

2.4 SYSTEM ARCHITECTURE

The `main()` shows the menu and calls one function per option. The operations share helper routines for safe input (`readLine()`, `readInt()`, `isValidNumber()`), searching and slot allotment (`findVehicle()`, `findFreeSlot()`), billing (`computeHours()`, `rateFor()`) and file storage (`saveVehicles()`, `loadVehicles()`). `displayVehicles()` and `showSummary()` only walk the list from `head`. The list in memory is the working copy and `parking.dat` is the permanent copy.

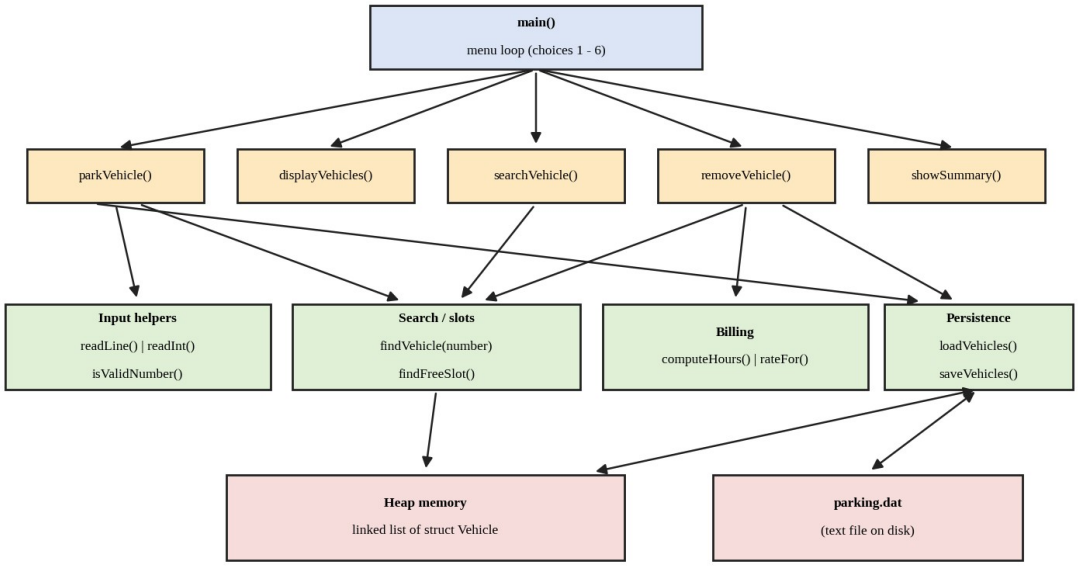


Fig. 2.2: Modular architecture of the system

2.5 FILE STORAGE

Each vehicle occupies five lines in `parking.dat`: slot number, vehicle type, entry time (seconds since 1 January 1970), vehicle number and owner name. The owner name comes last so it may contain spaces. On start-up `loadVehicles()` accepts a record only if the slot is between 1 and 20 and not already used, the type is 2 or 4, the entry time is positive, the vehicle number is valid and unique, and the owner name is not blank; otherwise the record is discarded.

2.6 PARKING CHARGES

The charge depends on the vehicle type and on the time parked. The time between entry and exit is rounded up to the next whole hour, and every vehicle pays for at least one hour.

Table 2.2: Parking charges

Vehicle type	Type code	Charge
Two wheeler	2	Rs 10 per hour
Four wheeler	4	Rs 20 per hour

2.7 ALGORITHMS

- **Park:** validate the number, owner and type; refuse if 20 vehicles are already inside; find the lowest unused slot; allocate a node, walk to the tail and link it, then save the file.
- **Search:** `findVehicle()` scans from `head` and returns the first node with the given vehicle number, or NULL.
- **Remove and bill:** walk with `temp` and `previous`; compute the hours and the bill; set `head` or `previous->next` to `temp->next`; free the node; save the file.
- **Slot allotment:** `findFreeSlot()` tries slot numbers 1 to 20 and returns the first one that no node uses, so the slot of a vehicle that has left is reused.

2.8 INPUT VALIDATION

Input is read line by line with `fgets()`, and excess characters are discarded to prevent buffer overflow. `readInt()` accepts a line only if `sscanf` converts exactly one integer and changes the target variable only on success. Vehicle numbers are converted to upper case so that up16ab1234 and UP16AB1234 are the same vehicle. End of input is treated as Exit, `malloc()` results are always checked, and duplicate numbers, numbers with symbols, empty owner names, invalid vehicle types, parking in a full lot and removing an unknown vehicle are refused.

2.9 COMPLEXITY

Table 2.3: Complexity of the operations (n = number of parked vehicles, S = 20 slots)

Operation	Time	Reason
Park vehicle	O(n·S)	Duplicate check, search for a free slot (S slots × n nodes) and walk to tail
Display / Save / Summary	O(n)	One visit per node
Search	O(n)	Linear scan with `findVehicle()`
Remove and bill	O(n)	Scan; unlinking is O(1)
Space	O(n)	One fixed-size node per parked vehicle

Since S is fixed at 20 and n can never exceed 20, every operation finishes instantly on any computer.

CHAPTER 3
IMPLEMENTATION

3.1 SAFE INPUT

`readInt()` reads a whole line and accepts it only if exactly one integer was converted, so entries such as 12abc are rejected. The result is copied to the caller's variable only when the conversion succeeds.

```
int readInt(const char *prompt, int *value)
{
    char line[100];
    char extra;
    int number;

    if (!readLine(prompt, line, sizeof(line)))
        return 0;

    if (sscanf(line, " %d %c", &number, &extra) != 1)
    {
        printf("Please enter a valid whole number.\n");
        return -1;
    }

    *value = number;
    return 1;
}
```

3.2 MENU AND PARKING A VEHICLE

`main()` loads saved data and repeats the menu until option 6. `parkVehicle()` checks that the lot is not full, validates the vehicle number, owner name and type, allots the lowest free slot, appends the new node to the tail and saves the file. A node is allocated only after all input is valid, so no memory has to be released on a failed entry.

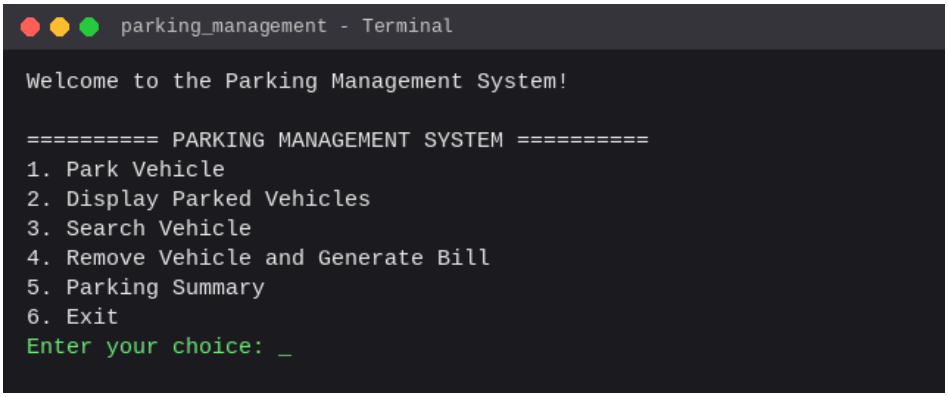


Fig. 3.1: Start-up screen showing the main menu

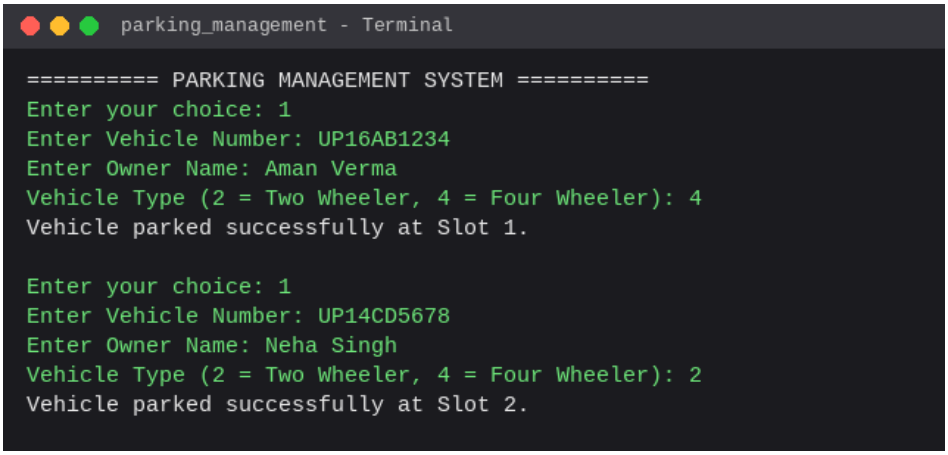


Fig. 3.2: Parking vehicles

3.3 DISPLAY AND SEARCH

`displayVehicles()` prints one row per node in the order of arrival; `%-20.20s` keeps the columns aligned by cutting long owner names on screen only. `searchVehicle()` converts the entered number to upper case, calls `findVehicle()` and prints every field. In Fig. 3.4 the first number was typed in lower case and was still found.

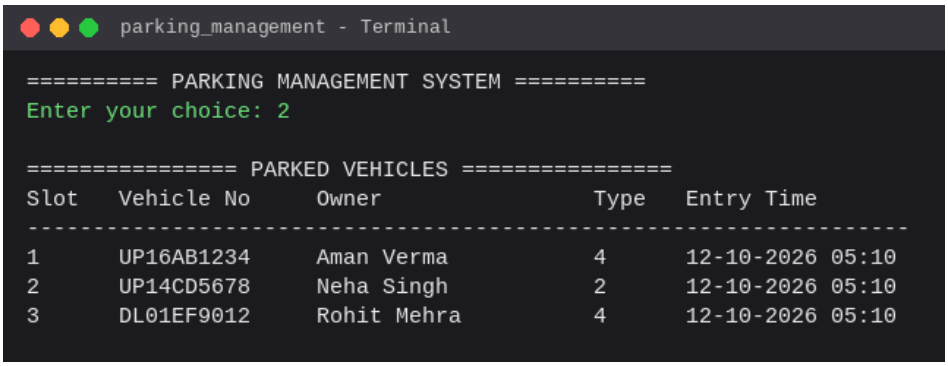


Fig. 3.3: Displaying all parked vehicles

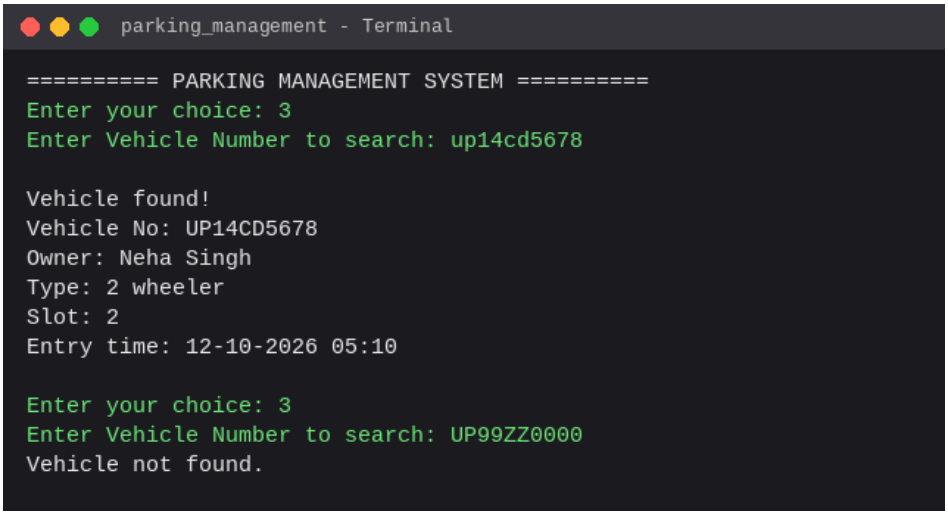


Fig. 3.4: Searching for an existing and a missing vehicle

3.4 REMOVING A VEHICLE AND BILLING

`removeVehicle()` finds the vehicle with two pointers, prints the bill, unlinks and frees the node and saves the file. `computeHours()` rounds the parked time up to whole hours with a minimum of one hour; the bill is the hours multiplied by the rate for the vehicle type.

```
int computeHours(time_t entry, time_t now)
{
    double seconds = difftime(now, entry);
    long hours;

    if (seconds < 0)
        seconds = 0;

    hours = (long)(seconds / SECONDS_PER_HOUR);
    if ((double)hours * SECONDS_PER_HOUR < seconds)
        hours++;              /* round up */
    if (hours < 1)
        hours = 1;            /* minimum one hour */

    return (int)hours;
}
```

To show a realistic bill without waiting, the entry time of the first vehicle in `parking.dat` was moved back by 2 hours 10 minutes before the run in Fig. 3.5, so it was charged for 3 hours. The second bill in the same figure is for a two wheeler that stayed for less than a minute and therefore pays the one-hour minimum.

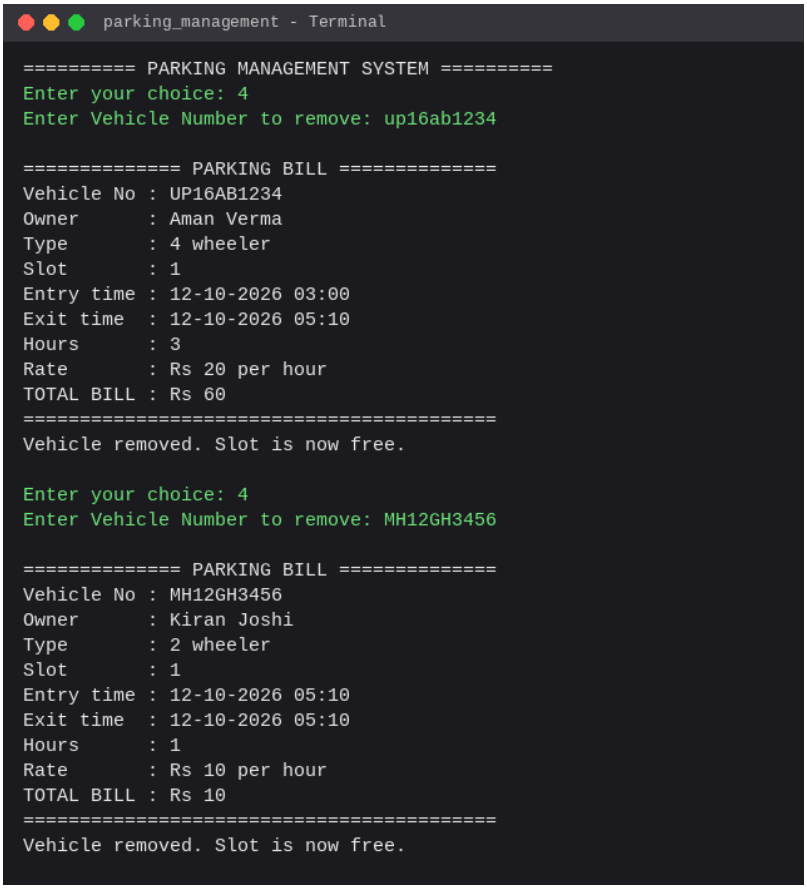


Fig. 3.5: Removing vehicles and generating bills

`showSummary()` counts the vehicles by type. When all 20 slots are taken, `parkVehicle()` refuses a new vehicle. Fig. 3.6 also shows that the slot freed by a removed vehicle is given to the next vehicle that arrives.

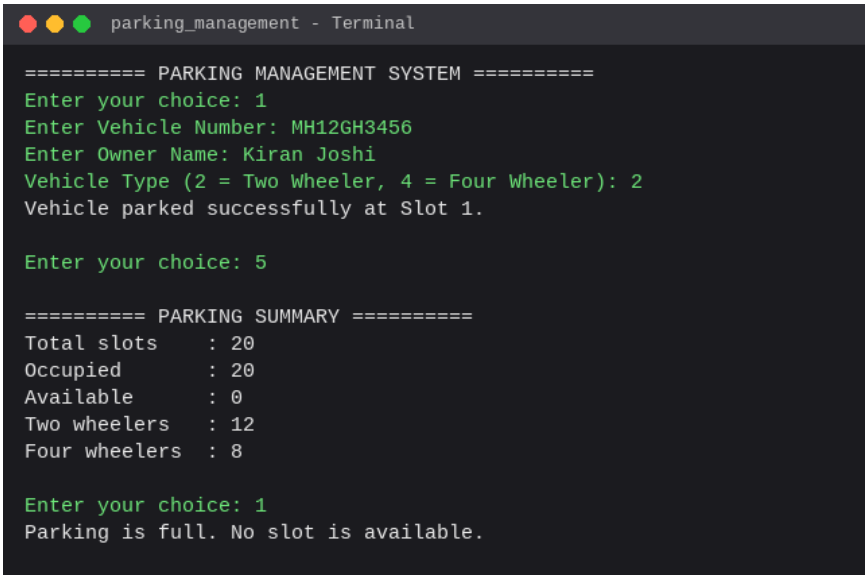


Fig. 3.6: Slot reuse, full parking lot and summary

3.5 ERROR HANDLING

Invalid choices, non-numeric input, duplicate vehicle numbers, symbols in a number, blank owner names, invalid vehicle types and unknown vehicle numbers all produce a clear message and leave the data unchanged.

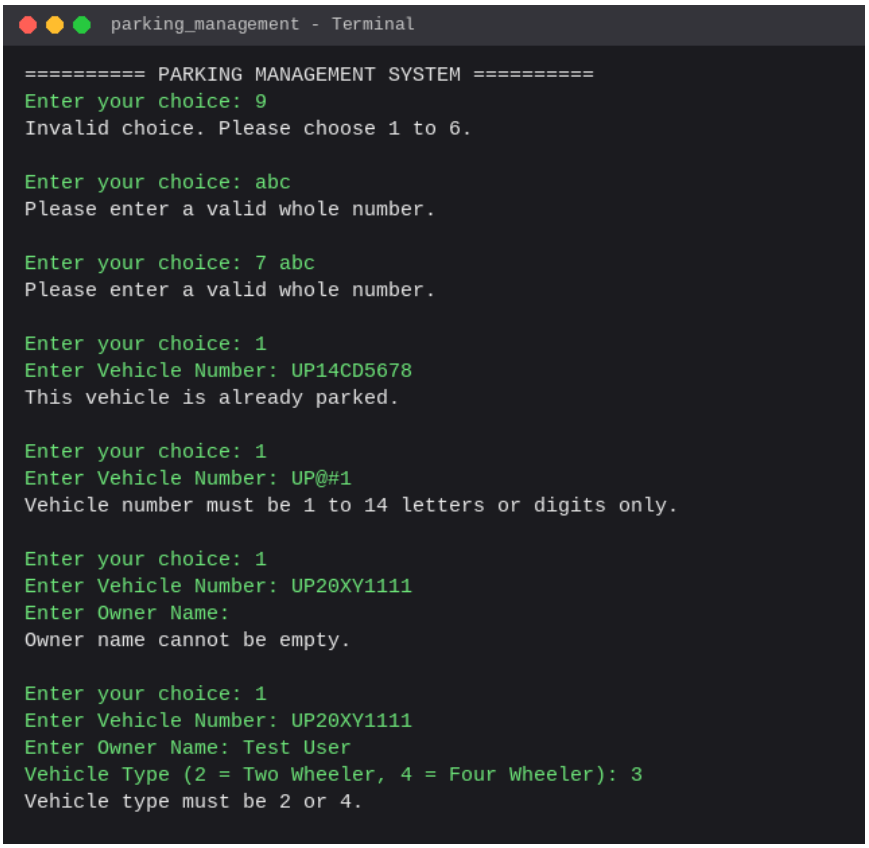


Fig. 3.7: Messages produced by invalid input

CHAPTER 4
RESULTS AND DISCUSSION

4.1 TEST CASES AND RESULTS

The program was tested with normal, boundary and invalid inputs, and complete sessions were also run under AddressSanitizer and UndefinedBehaviorSanitizer with no errors or memory leaks reported.

Table 4.1: Test cases and results

No.	Test case (input)	Expected and observed result	Result
T1	Park UP16AB1234 (four wheeler)	“Vehicle parked successfully at Slot 1.”	Pass
T2	Park UP14CD5678 (two wheeler) and DL01EF9012 (four wheeler)	Slots 2 and 3; appended in order of arrival	Pass
T3	Display all vehicles	Table with slot, number, owner, type and entry time	Pass
T4	Search up14cd5678 (lower case)	Vehicle found; all details shown	Pass
T5	Search UP99ZZ0000	“Vehicle not found.”	Pass
T6	Park UP14CD5678 again	“This vehicle is already parked.”	Pass
T7	Park with number UP@#1	“Vehicle number must be 1 to 14 letters or digits only.”	Pass
T8	Park with a blank owner name	“Owner name cannot be empty.”	Pass
T9	Vehicle type 3	“Vehicle type must be 2 or 4.”	Pass
T10	Vehicle type abc	“Please enter a valid whole number.”	Pass
T11	Menu choice 9	“Invalid choice. Please choose 1 to 6.”	Pass
T12	Menu choice abc, then 7 abc	“Please enter a valid whole number.”; program keeps running	Pass
T13	Remove a four wheeler parked for 2 h 10 min*	Bill: 3 hours × Rs 20 = Rs 60	Pass
T14	Remove a two wheeler parked for under 1 minute	Minimum charge: 1 hour, Rs 10	Pass
T15	Remove a two wheeler parked for 1 h 0 min 1 s*	Rounded up: 2 hours, Rs 20	Pass
T16	Remove UP00NONE00	“Vehicle not found.”	Pass
T17	Park a new vehicle after slot 1 was freed	Allotted Slot 1 (lowest free slot)	Pass
T18	Fill all 20 slots, then park a 21st vehicle	“Parking is full. No slot is available.”	Pass

No.	Test case (input)	Expected and observed result	Result
T19	Summary with 20 vehicles parked	Occupied 20, Available 0, two wheelers 12, four wheelers 8	Pass
T20	Exit and restart; also a record with slot 99 in `parking.dat`	Vehicles, slots and entry times reloaded; invalid record discarded	Pass

** The entry time in `parking.dat` was moved back by the stated time so that the test did not have to wait.*

4.2 DISCUSSION

All 20 tests passed. Vehicles, slots and entry times were reloaded correctly after every restart, a freed slot was reused by the next vehicle, and the lot refused a 21st vehicle when it was full. Because the number of free slots is computed from the list instead of being stored, the counts on the screen always agreed with the vehicles shown. Complete sessions run with the compiler's AddressSanitizer and UndefinedBehaviorSanitizer reported nothing.

Limitations:

- Search is by vehicle number only and is O(n); each addition walks to the tail and every change rewrites the whole file.
- The lot has a fixed 20 slots and two vehicle types; a slot is not matched to the size of the vehicle.
- Bills are shown on the screen only; no history of past bills or daily collection is kept.
- Billing depends on the computer's clock; if the clock is changed while a vehicle is parked, the bill will be wrong.
- Owner names are limited to 49 characters and vehicle numbers to 14; the table shows only the first 20 characters of an owner name.
- Records in `parking.dat` that fail validation at start-up are discarded without a message, and the text file can be edited by anyone.

CHAPTER 5
CONCLUSION

The project implemented a menu-driven Parking Management System in C using a singly linked list. All planned features work and all test cases passed. It strengthened understanding of pointers, dynamic memory, list traversal, file handling and complexity analysis, and showed how much effort goes into input validation and keeping stored data consistent.

Future scope:

- Add a tail pointer for O(1) insertion and a hash table or sorted structure for faster search.
- Keep separate zones or levels for two and four wheelers.
- Save every bill in a history file and print a daily collection report.
- Add monthly passes, an administrator password and fines for lost tickets.
- Use a binary file or database and add a graphical interface.

REFERENCES

1. Reema Thareja, Data Structures Using C, Oxford University Press.
2. Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, Prentice Hall.
3. Thomas H. Cormen et al., Introduction to Algorithms, MIT Press.
4. Yashavant Kanetkar, Let Us C, BPB Publications.
5. United Nations, The 17 Sustainable Development Goals, https://sdgs.un.org/goals.

APPENDIX: SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <time.h>

#define MAX_SLOTS 20
#define NUMBER_LEN 15
#define OWNER_LEN 50
#define DATA_FILE "parking.dat"
#define SECONDS_PER_HOUR 3600
#define RATE_2W 10
#define RATE_4W 20

struct Vehicle
{
    char number[NUMBER_LEN];
    char owner[OWNER_LEN];
    int type;
    int slot;
    time_t entryTime;
    struct Vehicle *next;
};

struct Vehicle *head = NULL;

/* Read a line of text safely. Returns 0 if input ends. */
int readLine(const char *prompt, char *buffer, size_t size)
{
    int ch;
    size_t len;

    printf("%s", prompt);
    if (fgets(buffer, (int)size, stdin) == NULL)
        return 0;

    len = strlen(buffer);
    if (len > 0 && buffer[len - 1] == '\n')
    {
        buffer[len - 1] = '\0';
    }
    else
    {
        while ((ch = getchar()) != '\n' && ch != EOF)
        {
            /* Discard extra characters. */
        }
    }
    return 1;
}

/* Read an integer and reject non-integer input. */
int readInt(const char *prompt, int *value)
{
    char line[100];
```

```
    char extra;
    int number;

    if (!readLine(prompt, line, sizeof(line)))
        return 0;

    if (sscanf(line, " %d %c", &number, &extra) != 1)
    {
        printf("Please enter a valid whole number.\n");
        return -1;
    }

    *value = number;
    return 1;
}

int isBlank(const char *text)
{
    while (*text != '\0')
    {
        if (!isspace((unsigned char)*text))
            return 0;
        text++;
    }
    return 1;
}

/* A vehicle number must be 1 to 14 letters or digits. */
int isValidNumber(const char *text)
{
    size_t i, len = strlen(text);

    if (len == 0 || len >= NUMBER_LEN)
        return 0;
    for (i = 0; i < len; i++)
    {
        if (!isalnum((unsigned char)text[i]))
            return 0;
    }
    return 1;
}

void toUpperCase(char *text)
{
    for (; *text != '\0'; text++)
        *text = (char)toupper((unsigned char)*text);
}

struct Vehicle *findVehicle(const char *number)
{
    struct Vehicle *temp = head;

    while (temp != NULL)
    {
        if (strcmp(temp->number, number) == 0)
            return temp;
        temp = temp->next;
    }
```

```
    return NULL;
}

int slotInUse(int slot)
{
    struct Vehicle *temp = head;

    while (temp != NULL)
    {
        if (temp->slot == slot)
            return 1;
        temp = temp->next;
    }
    return 0;
}

int countVehicles(void)
{
    int count = 0;
    struct Vehicle *temp = head;

    while (temp != NULL)
    {
        count++;
        temp = temp->next;
    }
    return count;
}

/* Lowest free slot number, or 0 if the lot is full. */
int findFreeSlot(void)
{
    int slot;

    for (slot = 1; slot <= MAX_SLOTS; slot++)
    {
        if (!slotInUse(slot))
            return slot;
    }
    return 0;
}

/* Hours are rounded up; the minimum charge is one hour. */
int computeHours(time_t entry, time_t now)
{
    double seconds = difftime(now, entry);
    long hours;

    if (seconds < 0)
        seconds = 0;

    hours = (long)(seconds / SECONDS_PER_HOUR);
    if ((double)hours * SECONDS_PER_HOUR < seconds)
        hours++;
    if (hours < 1)
        hours = 1;

    return (int)hours;
```

```
}

int rateFor(int type)
{
    return (type == 2) ? RATE_2W : RATE_4W;
}

void formatTime(time_t value, char *buffer, size_t size)
{
    struct tm *info = localtime(&value);

    if (info == NULL || strftime(buffer, size, "%d-%m-%Y %H:%M", info) == 0)
        snprintf(buffer, size, "unknown");
}

void saveVehicles(void)
{
    FILE *file = fopen(DATA_FILE, "w");
    struct Vehicle *temp = head;

    if (file == NULL)
    {
        printf("Warning: Could not save parking data.\n");
        return;
    }

    while (temp != NULL)
    {
        /* slot, type, entry time, number, owner */
        fprintf(file, "%d\n%d\n%lld\n%s\n%s\n",
                temp->slot,
                temp->type,
                (long long)temp->entryTime,
                temp->number,
                temp->owner);
        temp = temp->next;
    }
    fclose(file);
}

void loadVehicles(void)
{
    FILE *file = fopen(DATA_FILE, "r");
    struct Vehicle *newVehicle, *tail = NULL;
    int slot, type;
    long long entry;

    if (file == NULL)
        return; /* First run: no saved file yet. */

    while (1)
    {
        if (fscanf(file, "%d\n%d\n%lld\n", &slot, &type, &entry) != 3)
            break;

        newVehicle = (struct Vehicle *)malloc(sizeof(struct Vehicle));
        if (newVehicle == NULL)
        {
```

```
            printf("Warning: Not enough memory to load all vehicles.\n");
            break;
        }

        if (fgets(newVehicle->number, NUMBER_LEN, file) == NULL ||
            fgets(newVehicle->owner, OWNER_LEN, file) == NULL)
        {
            free(newVehicle);
            break;
        }

        newVehicle->number[strcspn(newVehicle->number, "\r\n")] = '\0';
        newVehicle->owner[strcspn(newVehicle->owner, "\r\n")] = '\0';
        newVehicle->slot = slot;
        newVehicle->type = type;
        newVehicle->entryTime = (time_t)entry;
        newVehicle->next = NULL;

        if (slot < 1 || slot > MAX_SLOTS ||
            (type != 2 && type != 4) ||
            entry <= 0 ||
            !isValidNumber(newVehicle->number) ||
            isBlank(newVehicle->owner) ||
            findVehicle(newVehicle->number) != NULL ||
            slotInUse(slot))
        {
            free(newVehicle);
            continue;
        }

        if (head == NULL)
            head = newVehicle;
        else
            tail->next = newVehicle;
        tail = newVehicle;
    }
    fclose(file);
}

void parkVehicle(void)
{
    struct Vehicle *newVehicle, *temp;
    char number[100], owner[100];
    int type, result, slot;

    if (countVehicles() >= MAX_SLOTS)
    {
        printf("Parking is full. No slot is available.\n");
        return;
    }

    if (!readLine("Enter Vehicle Number: ", number, sizeof(number)))
        return;
    toUpperCase(number);
    if (!isValidNumber(number))
    {
        printf("Vehicle number must be 1 to 14 letters or digits only.\n");
        return;
```

```
    }
    if (findVehicle(number) != NULL)
    {
        printf("This vehicle is already parked.\n");
        return;
    }

    if (!readLine("Enter Owner Name: ", owner, sizeof(owner)) ||
        isBlank(owner))
    {
        printf("Owner name cannot be empty.\n");
        return;
    }
    if (strlen(owner) >= OWNER_LEN)
    {
        printf("Owner name must be less than %d characters.\n", OWNER_LEN);
        return;
    }

    result = readInt("Vehicle Type (2 = Two Wheeler, 4 = Four Wheeler): ", &type);
    if (result != 1)
        return;
    if (type != 2 && type != 4)
    {
        printf("Vehicle type must be 2 or 4.\n");
        return;
    }

    slot = findFreeSlot();
    newVehicle = (struct Vehicle *)malloc(sizeof(struct Vehicle));
    if (newVehicle == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    strcpy(newVehicle->number, number);
    strcpy(newVehicle->owner, owner);
    newVehicle->type = type;
    newVehicle->slot = slot;
    newVehicle->entryTime = time(NULL);
    newVehicle->next = NULL;

    if (head == NULL)
    {
        head = newVehicle;
    }
    else
    {
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newVehicle;
    }

    saveVehicles();
    printf("Vehicle parked successfully at Slot %d.\n", slot);
}
```

```
void displayVehicles(void)
{
    struct Vehicle *temp = head;
    char when[40];

    if (head == NULL)
    {
        printf("No vehicles are currently parked.\n");
        return;
    }

    printf("\n================ PARKED VEHICLES ================\n");
    printf("%-6s %-14s %-20s %-6s %-16s\n",
           "Slot", "Vehicle No", "Owner", "Type", "Entry Time");
    printf("-----------------------------------------------------------------------\n");

    while (temp != NULL)
    {
        formatTime(temp->entryTime, when, sizeof(when));
        printf("%-6d %-14s %-20.20s %-6d %-16s\n",
               temp->slot, temp->number, temp->owner, temp->type, when);
        temp = temp->next;
    }
}

void searchVehicle(void)
{
    char number[100];
    struct Vehicle *vehicle;
    char when[40];

    if (!readLine("Enter Vehicle Number to search: ", number, sizeof(number)))
        return;
    toUpperCase(number);

    vehicle = findVehicle(number);
    if (vehicle == NULL)
    {
        printf("Vehicle not found.\n");
        return;
    }

    formatTime(vehicle->entryTime, when, sizeof(when));
    printf("\nVehicle found!\n");
    printf("Vehicle No: %s\n", vehicle->number);
    printf("Owner: %s\n", vehicle->owner);
    printf("Type: %d wheeler\n", vehicle->type);
    printf("Slot: %d\n", vehicle->slot);
    printf("Entry time: %s\n", when);
}

void removeVehicle(void)
{
    struct Vehicle *temp = head, *previous = NULL;
    char number[100], entry[40], leave[40];
    time_t now;
    int hours, rate;
```

```
    if (!readLine("Enter Vehicle Number to remove: ", number, sizeof(number)))
        return;
    toUpperCase(number);

    while (temp != NULL && strcmp(temp->number, number) != 0)
    {
        previous = temp;
        temp = temp->next;
    }

    if (temp == NULL)
    {
        printf("Vehicle not found.\n");
        return;
    }

    now = time(NULL);
    hours = computeHours(temp->entryTime, now);
    rate = rateFor(temp->type);
    formatTime(temp->entryTime, entry, sizeof(entry));
    formatTime(now, leave, sizeof(leave));

    printf("\n============== PARKING BILL ==============\n");
    printf("Vehicle No : %s\n", temp->number);
    printf("Owner      : %s\n", temp->owner);
    printf("Type       : %d wheeler\n", temp->type);
    printf("Slot       : %d\n", temp->slot);
    printf("Entry time : %s\n", entry);
    printf("Exit time  : %s\n", leave);
    printf("Hours      : %d\n", hours);
    printf("Rate       : Rs %d per hour\n", rate);
    printf("TOTAL BILL : Rs %d\n", hours * rate);
    printf("==========================================\n");

    if (previous == NULL)
        head = temp->next;
    else
        previous->next = temp->next;
    free(temp);

    saveVehicles();
    printf("Vehicle removed. Slot is now free.\n");
}

void showSummary(void)
{
    struct Vehicle *temp = head;
    int two = 0, four = 0, used;

    while (temp != NULL)
    {
        if (temp->type == 2)
            two++;
        else
            four++;
        temp = temp->next;
    }
```

```
    used = two + four;

    printf("\n========== PARKING SUMMARY ==========\n");
    printf("Total slots    : %d\n", MAX_SLOTS);
    printf("Occupied       : %d\n", used);
    printf("Available      : %d\n", MAX_SLOTS - used);
    printf("Two wheelers   : %d\n", two);
    printf("Four wheelers  : %d\n", four);
}

void freeList(void)
{
    struct Vehicle *temp;

    while (head != NULL)
    {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main(void)
{
    int choice = 0, result;

    loadVehicles();
    printf("Welcome to the Parking Management System!\n");

    do
    {
        printf("\n========== PARKING MANAGEMENT SYSTEM ==========\n");
        printf("1. Park Vehicle\n");
        printf("2. Display Parked Vehicles\n");
        printf("3. Search Vehicle\n");
        printf("4. Remove Vehicle and Generate Bill\n");
        printf("5. Parking Summary\n");
        printf("6. Exit\n");
        result = readInt("Enter your choice: ", &choice);

        if (result == 0)
            choice = 6;
        else if (result == -1)
            continue;

        switch (choice)
        {
        case 1:
            parkVehicle();
            break;
        case 2:
            displayVehicles();
            break;
        case 3:
            searchVehicle();
            break;
        case 4:
            removeVehicle();
```

```
            break;
        case 5:
            showSummary();
            break;
        case 6:
            saveVehicles();
            freeList();
            printf("Parking data saved. Goodbye!\n");
            break;
        default:
            printf("Invalid choice. Please choose 1 to 6.\n");
        }
    } while (choice != 6);

    return 0;
}
```